16

Лекция 22–23 декабря

Сжатие и границы

Константин Чухарев

Дискретная математика — Осень 2026

Сжатие и границы

> *Основная задача связи состоит в точном или приближенном воспроизведении в некотором месте сообщения, выбранного для передачи в другом месте.*
>
> **— Клод Шеннон**

Защита и сжатие

Код Хэмминга защищает от ошибок, добавляя избыточность.

Код Хаффмана решает обратную задачу и сжимает данные без потерь.

Частым символам — короткие кодовые слова, редким — длинные.

Сжатие здесь — без потерь, и распакованный файл совпадает с исходным бит в бит.

С потерями живут картинки и звук — у них другая норма качества и другая математика.

Азбука Морзе тоже берёт неравные длины, но там нужны паузы, ведь E — одна точка, а I — две.

Префиксный код

Слова разной длины полезны, только если поток читается без пауз и возвратов. Этого добиваются одним запретом.

Определение 1 (Префиксный код)

Код называется **префиксным**, если ни одно кодовое слово не является началом другого.

Пример

Код $\{0, 10, 11\}$ префиксный, потому что ни одно слово не является началом другого.

Код $\{0, 01, 011\}$ префиксным не является. Слово 0 может продолжиться как 01 или как 011, и декодер не знает, кончилось ли слово.

На дереве кода запрет означает, что кодовые слова — листья, и ни одно не стоит на пути к другому.

Декодирование без задержки

Пример

Слова $A = 0$, $B = 10$, $C = 110$, $D = 1110$, $E = 1111$.

Поток `1101111` читается слева направо, и `110` — это C, а сразу за ним `1111` — это E.

Другого разбиения нет, потому что конец слова виден в момент прочтения.

Префиксность снимает задержку, и разбор выходит единственным.

Префиксные коды называют *мгновенными* (англ. instantaneous), потому что символ опознаётся сразу по прочтении своего слова.

Тем самым декодер работает в реальном времени — задержка ограничена длиной одного слова.

Неравенство Крафта

Не всякий набор длин годится, ведь слово длины 1 занимает половину кодового дерева.

Теорема 1 (Неравенство Крафта)

Если бинарный префиксный код состоит из слов с длинами $l_1, ..., l_n$, то

$$\sum_{i=1}^{n} 2^{-l_i} \leq 1.$$

Доказательство (подсчётом строк)

Докажем оценку продолжениями слов до общей длины $L = \max_i l_i$.

Каждое слово продолжается до 2^{L-l_i} строк длины L, и из-за префиксности продолжения разных слов не пересекаются.

Все они среди 2^L строк, значит, $\sum_i 2^{L-l_i} \leq 2^L$ — деление на 2^L даёт неравенство.

Пример Крафта

Пример (Дерево глубины 2)

Возьмём длины $2, 2, 2, 2$. Сумма равна $4 \cdot \frac{1}{4} = 1$, и четыре листа заполняют дерево глубины 2 без свободных мест.

Пятое слово длины 2 дало бы $5 \cdot \frac{1}{4} = \frac{5}{4} > 1$, и места в дереве уже нет.

Для пяти символов годится длина 3, и сумма $5 \cdot \frac{1}{8} = \frac{5}{8} < 1$ оставляет три свободных листа на глубине 3.

Алгоритм Хаффмана

Алгоритм жадный и строит дерево снизу вверх.

Пример

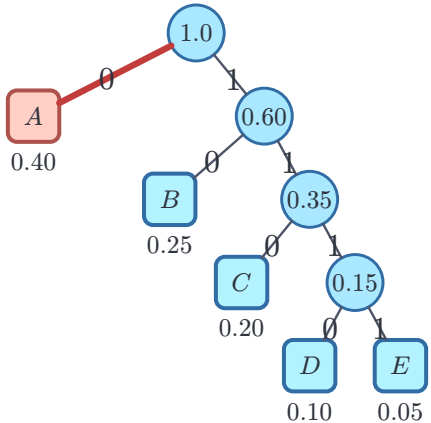
Возьмём частоты $A = 0.40$, $B = 0.25$, $C = 0.20$, $D = 0.10$, $E = 0.05$.

1. Сливаем два символа наименьшего веса, D и E, в узел 0.15.
2. Дальше C и DE — узел 0.35.
3. Дальше B и CDE — узел 0.60.
4. Наконец A и BCDE — корень 1.00.

Реализация — очередь с минимальным весом, где на шаг идут два извлечения, одно слияние и одна вставка, а всего шагов $n - 1$.

Тяжёлые символы сливаются последними и остаются у корня — потому и получают короткие слова.

Дерево Хаффмана



Путь от корня к листу — кодовое слово, левый переход даёт 0, правый — 1.

Во внутренних узлах стоит сумма частот поддерева, поэтому корень несёт 1.00.

Средняя длина

Определение 2 (Средняя длина кода)

Средней длиной кода называют величину

$$L = \sum_i f_i l_i,$$

где f_i — частота символа, l_i — длина его слова.

Пример

$$L = 0.40 \cdot 1 + 0.25 \cdot 2 + 0.20 \cdot 3 + 0.10 \cdot 4 + 0.05 \cdot 4 = 2.10$$

бита на символ — против $\lceil \log_2 5 \rceil = 3$ бита у равномерного кода на пять символов.

При равных частотах и n, равном степени двойки, код Хаффмана равномерен, а выигрыш появляется только при неравных частотах.

Повторения и словарное сжатие

Хаффман видит только частоты, а главный резерв реальных данных — повторения.

Словарное сжатие заменяет повтор ссылкой, и канал передаёт адрес, а не копию.

Определение 3 (Тройка LZ77)

Тройкой LZ77 (o, l, a) называют запись повтора из смещения назад, длины совпадения и первого несовпавшего символа.

Кодер ищет совпадения в *скользящем окне* — уже закодированном хвосте входа.

Если совпадение доходит до конца входа, третья компонента не нужна и остаётся пара (o, l).

DEFLATE (gzip, zip, PNG) — LZ77 плюс код Хаффмана, где LZ выучивает повторения, а Хаффман ужимает остаток.

Тройки LZ77 на «ABRACADABRA»

Окно — весь уже закодированный префикс, и совпадение выбирается самое длинное.

Тройка	Совпало	Свежий символ
$(0, 0, A)$	$-$	A
$(0, 0, B)$	$-$	B
$(0, 0, R)$	$-$	R
$(3, 1, C)$	A	C
$(2, 1, D)$	A	D
$(7, 4)$	ABRA	конец входа

Пример

К моменту последней записи декодер уже вывел `ABRACAD`, и пара $(7, 4)$ переписывает из него `ABRA`.

Распаковка троек LZ77

Декодер идёт по тройкам и берёт копию из уже выведенного текста.

Тройка	Выведено до	Копия	Выведено после
$(0, 0, A)$	пусто	—	A
$(0, 0, B)$	A	—	AB
$(0, 0, R)$	AB	—	ABR
$(3, 1, C)$	ABR	A	ABRAC
$(2, 1, D)$	ABRAC	A	ABRACAD
$(7, 4)$	ABRACAD	ABRA	ABRACADABRA

Копия читается посимвольно и может заходить на дописываемый участок. Тройка $(1, 3, A)$ после буквы A даёт пять букв A, ведь копирование идёт вперёд и захватывает только что записанные буквы.

Энтропия

Определение 4 (Энтропия)

Для независимых символов с вероятностями $p_1, \ldots, p_n$ энтропия равна

$$H = -\sum_{i=1}^{n} p_i \log_2 p_i.$$

Пример (Энтропия на числах)

У равновероятной пары оба слагаемых равны 0.5, и $H = 1$ бит. У достоверного символа $\log_2 1 = 0$, и сообщать не о чем.

Частоты примера с пятью символами дают $H = 0.40 \cdot 1.322 + 0.25 \cdot 2 + 0.20 \cdot 2.322 + 0.10 \cdot 3.322 + 0.05 \cdot 4.322 \approx 2.04$ бита на символ, и код с $L = 2.10$ отстаёт от предела на шесть сотых.

Граница энтропии

Энтропия — нижний предел для любого префиксного кода, а Хаффман подходит к нему на расстояние меньше бита.

Теорема 2 (Шеннон, 1948 — источник)

Средняя длина любого префиксного кода не меньше энтропии, $L \geq H$.

Код Хаффмана укладывается в $H \leq L < H + 1$.

Пример

Для пяти символов $H \approx 2.04$, а код Хаффмана дал $L = 2.10$.

Граница обещает $2.04 \leq 2.10 < 3.04$, и до нижнего предела остаётся шесть сотых бита.

Равенство $L = H$ возможно только тогда, когда все p_i обратны степеням двойки, и длины $-\log_2 p_i$ целы.

Сжимаемость строк

У сжатия есть счётный предел, и архиватора, сжимающего любой файл, не существует.

Теорема 3 (Почти все строки несжимаемы)

Доля строк длины n, сжимаемых хотя бы на k битов, не превосходит 2^{1-k} при $k \leq n$.

Доказательство (подсчётом)

Докажем подсчётом прообразов у инъективного распаковщика.

Сжимаемые строки — образы строк длины до $n-k$, а таких строк $2^{n-k+1}-1$.

Распаковщик инъективен, поэтому образы не пересекаются, и доля не превышает $\frac{2^{n-k+1}-1}{2^n} < 2^{1-k}$.

Пример

При $k = 10$ сжимается меньше $2^{-9} \approx 0.2\%$ строк, это один файл из пятисот.

Почему сжатый файл выглядит случайным?

Файл из n равновероятных байтов имеет энтропию $8n$ бит. Сжатие без потерь не может сделать его короче.

Каждому файлу из n байтов отвечает строка из $8n$ битов, и таких строк 2^{8n}. Строк короче $8n$ битов меньше, чем 2^{8n}, поэтому хотя бы один файл не сжимается.

> Реальные данные — крошечное структурированное меньшинство. Хорошо сжатый файл выглядит как случайный шум, и повторное сжатие его не уменьшает.

Обратная теорема Крафта

Обратная теорема разрешает все наборы длин, которые проходит Крафт, поэтому длины можно выбирать раньше кода.

Теорема 4 (Обратная теорема Крафта)

Если $\sum_{i=1}^{n} 2^{-l_i} \leq 1$, то префиксный код с длинами $l_1, \ldots, l_n$ существует.

Доказательство (построением)

Упорядочим длины по возрастанию и возьмём словом i первые l_i битов двоичной записи суммы $s_i = \sum_{k<i} 2^{-l_k}$.

Корректность. Слову из l_i битов отвечает отрезок $\left[s_i, s_i + 2^{-l_i}\right)$, и это в точности строки с таким началом.

Соседние отрезки стыкуются, а неравенство Крафта держит их общий конец не дальше 1, поэтому вложенных отрезков нет и одно слово не служит началом другого.

Пример длин и дыры

Пример (Суммы: полная и с дырой)

Длины $(1, 2, 3, 3)$ дают слова 0, 10, 110, 111 по суммам $s_1 = 0$, $s_2 = \frac{1}{2} = 0.1_2$, $s_3 = \frac{3}{4} = 0.11_2$, $s_4 = \frac{7}{8} = 0.111_2$ — код $\{0, 10, 110, 111\}$, тот же, что строит Хаффман для частот $\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{8}$.

Длины $(1, 2)$ дают сумму $\frac{1}{2} + \frac{1}{4} = \frac{3}{4} < 1$, поэтому код $\{0, 10\}$ не заполняет дерево, и свободной остаётся вся ветка 11.

Для пяти слов $A = 0$, $B = 10$, $C = 110$, $D = 1110$, $E = 1111$ сумма равна $\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \frac{1}{16} = 1$, и дерево заполнено без дыр.

Строгое неравенство разрешает, но не требует заполнять всё. Дыру можно достроить новыми словами, и код останется префиксным.

Длины и префиксность

Неравенство необходимо для префиксного кода, но само по себе префиксности не даёт.

Сумма не делает код префиксным

Сумма для кода $\{0, 01, 011\}$ равна $\frac{1}{2} + \frac{1}{4} + \frac{1}{8} = \frac{7}{8} \leq 1$, но слово 0 — префикс слова 01.

Обратная теорема гарантирует лишь существование другого префиксного кода с теми же длинами, например $\{0, 10, 110\}$.

Однозначность декодирования слабее префиксности. Код $\{0, 01, 011\}$ не префиксный, но разбирается единственным образом, только с задержкой.

Теорема Макмиллана распространяет то же неравенство на любой однозначно декодируемый код, даже не префиксный.

Префиксность бесплатна, ведь класс кодов уже, а лучшие длины те же.

Доказательство границы энтропии

Нижняя оценка опирается на неравенство Крафта, верхняя — на округление длин вверх.

Доказательство (оценками по отдельности)

Нижняя. Для $q_i = 2^{-l_i}$ разность $L - H = \sum_i p_i \log_2(p_i/q_i)$ неотрицательна.

По неравенству о сумме логарифмов она не меньше $\log_2\left(1/\sum_i q_i\right) \geq 0$, а $\sum_i q_i \leq 1$ — это неравенство Крафта.

Верхняя. Длины $l_i = \lceil -\log_2 p_i \rceil$ проходят неравенство Крафта, ведь $2^{-l_i} \leq p_i$, и обратная теорема даёт префиксный код.

Каждое $l_i < -\log_2 p_i + 1$, поэтому $L < H + 1$, а оптимальный код Хаффмана не хуже.

На блоках длины n зазор сжимается до $H \leq L < H + \frac{1}{n}$, и предел достигается сколь угодно близко.

Оптимальность через обмен

Теорема 5 (Обмен без ухудшения)

Среди оптимальных деревьев найдётся такое, где два редчайших символа — соседние листья максимальной глубины.

Доказательство (двумя обменами)

Пусть a, b — редчайшие символы, а x, y — соседние листья максимальной глубины l оптимального дерева.

Обмен a и x меняет среднюю длину на $(f_a - f_x)(l_x - l_a) \le 0$, ведь $f_a \le f_x$ и $l_x \le l_a$.

Тот же обмен ставит b на место y, и редчайшие символы оказываются соседями на максимальной глубине.

В оптимальном дереве пяти символов редчайшие $D = 0.10$ и $E = 0.05$ уже стоят на глубине 4, поэтому обмен не меняет длину.

Оптимальность по индукции

Теорема 6 (Оптимальность кода Хаффмана)

Для заданных частот код Хаффмана имеет минимальную среднюю длину среди всех префиксных кодов.

Доказательство (индукцией по числу символов n)

База. При $n = 1$ длина $L = 0$ — короче невозможно.

Шаг. По обменной теореме есть оптимальное дерево, где редчайшие a, b — соседи максимальной глубины.

Склейка их в лист веса $f_a + f_b$ уменьшает L ровно на $f_a + f_b$. По индукции алгоритм оптимален для $n - 1$ символов, а разворачивание листа прибавляет те же $f_a + f_b$ любому коду — дерево оптимально и для n.

Хаффман для шести символов

Возьмём частоты $A = 0.30$, $B = 0.25$, $C = 0.15$, $D = 0.12$, $E = 0.10$, $F = 0.08$.

Шаг	Слияние	Вес узла	Лес после шага
1	$F + E$	0.18	$0.12, 0.15, 0.18, 0.25, 0.30$
2	$D + C$	0.27	$0.18, 0.25, 0.27, 0.30$
3	$\mathrm{FE} + B$	0.43	$0.27, 0.30, 0.43$
4	$\mathrm{DC} + A$	0.57	$0.43, 0.57$
5	$\mathrm{FEB} + \mathrm{DCA}$	1.00	корень

Пять слияний собирают дерево, и в каждой паре меньший вес получает ребро 0, а больший — ребро 1, поэтому редчайшие F и E уходят на дно, а частые A и B поднимаются к корню.

Код и средняя длина для шести символов

Символ	Частота	Код	Длина l_i
A	0.30	11	2
B	0.25	01	2
C	0.15	101	3
D	0.12	100	3
E	0.10	001	3
F	0.08	000	3

Пример

$$L = 0.30 \cdot 2 + 0.25 \cdot 2 + 0.45 \cdot 3 = 2.45,$$

бита на символ против 3 у равномерного кода, экономия 18%.

Энтропия тех же частот $H \approx 2.42$ лежит в трёх сотых ниже.

Почему 2.45 минимальна?

Пример (Конкуренты кода Хаффмана)

Возьмём длины $2, 3, 3, 3, 3, 3$. Крафт выполнен, ведь $\frac{1}{4} + 5 \cdot \frac{1}{8} = \frac{7}{8} \leq 1$, но средняя длина равна $0.30 \cdot 2 + 0.70 \cdot 3 = 2.70$.

Длины $2, 2, 3, 3, 3, 3$ дают сумму $\frac{1}{4} + \frac{1}{4} + 4 \cdot \frac{1}{8} = 1$ и среднюю длину $0.30 \cdot 2 + 0.25 \cdot 2 + 0.45 \cdot 3 = 2.45$, ровно как у Хаффмана.

Лучше 2.45 на этих частотах не бывает, и оба конкурента проигрывают или сравниваются с Хаффманом.

Код Шеннона — Фано

Шеннон и Фано строят код сверху вниз, деля алфавит на две почти равные половины.

Алгоритм Шеннона — Фано.

1. Выписать символы по убыванию частот.
2. Разделить список на две части так, чтобы суммы частот были как можно ближе.
3. Первой части приписать 0, второй — 1.
4. Рекурсивно повторить деление внутри каждой части.

Хаффман собирает дерево снизу вверх, от редких символов. Шеннон — Фано делит множество сверху вниз. В общем случае Шеннон — Фано не гарантирует оптимальность, хотя часто даёт тот же код.

Шеннон — Фано, пример

Возьмём те же частоты, что у Хаффмана. $A = 0.30$, $B = 0.25$, $C = 0.15$, $D = 0.12$, $E = 0.10$, $F = 0.08$.

Символ	Частота	Код	Длина l_i
A	0.30	00	2
B	0.25	01	2
C	0.15	100	3
D	0.12	101	3
E	0.10	110	3
F	0.08	111	3

Первый разрез идёт после B. Сумма $A + B = 0.55$ против 0.45. Второй разрез идёт после D. Сумма $C + D = 0.27$ против $E + F = 0.18$.

Шеннон — Фано, пример [2]

Пример

$$L = 0.30 \cdot 2 + 0.25 \cdot 2 + 0.45 \cdot 3 = 2.45.$$

На этих частотах Шеннон — Фано совпал с Хаффманом.

Кодер Хаффмана на Rust

Вся механика — список поддеревьев, отсортированный по весу. Извлекаем два лёгких поддерева, дописываем им по биту, возвращаем слияние.

```rust
#[derive(Clone)]
enum Node {
    Leaf(char, u32),
    Branch(u32, Box<Node>, Box<Node>),
}

fn weight(n: &Node) -> u32 {
    match n {
        Node::Leaf(_, w) | Node::Branch(w, _, _) => *w,
    }
}

fn huffman(freq: &[(char, u32)]) -> Node {
    let mut nodes: Vec<Node> =
        freq.iter().map(|&(c, w)| Node::Leaf(c, w)).collect();
```

Кодер Хаффмана на Rust [2]

```
    while nodes.len() > 1 {
        nodes.sort_by_key(weight);
        let b = nodes.remove(0);
        let a = nodes.remove(0);
        let w = weight(&a) + weight(&b);
        nodes.push(Node::Branch(w, Box::new(a), Box::new(b)));
    }
    nodes.pop().unwrap()
}

fn codes(node: &Node, prefix: String, out: &mut Vec<(char, String)>) {
    match node {
        Node::Leaf(c, _) => out.push((*c, prefix)),
        Node::Branch(_, l, r) => {
            codes(l, format!("{prefix}0"), out);
            codes(r, format!("{prefix}1"), out);
        }
```

Кодер Хаффмана на Rust [3]

```
    }
}
```

Выход кодера

Код возвращает список пар, и для «ABRACADABRA» он выглядит так.

Пример

```
R 2 000
B 2 001
D 1 010
C 1 011
A 5 1
total: 23 bits vs 88
```

Сумма длин даёт $5 \cdot 1 + 2 \cdot 3 + 2 \cdot 3 + 1 \cdot 3 + 1 \cdot 3 = 23$ бита против 88.

Нули и единицы на рёбрах свободны, поэтому коды могут отличаться от ручной таблицы, а в файл пишут таблицу длин вместо дерева.

Словарь фраз LZ78

Выход LZ78 — номер выученной фразы плюс новый символ.

Пара	Номер фразы	Новая фраза
$(0, A)$	1	A
$(0, B)$	2	B
$(0, R)$	3	R
$(1, C)$	4	AC
$(1, D)$	5	AD
$(1, B)$	6	AB
$(3, A)$	7	RA

Семь пар покрывают одиннадцать букв, и декодер строит словарь синхронно.

Получив $(1, C)$, он берёт фразу с номером 1, то есть A, и дописывает C.

Вариант LZW

Пример

LZW — LZ78 с алфавитом в стартовом словаре, и пара вырождается в чистый номер фразы.

При нумерации $A = 0$, $B = 1$, $C = 2$, $D = 3$, $R = 4$ слово «ABRACADABRA» даёт

$$0, 1, 4, 0, 2, 0, 3, 5, 7,$$

девять номеров, и по 4 бита на номер это 36 битов против 88 исходных.

Номера коротки, потому что словарь растёт медленно, и после k выученных фраз максимальный номер $|\Sigma| + k - 1$ укладывается в $\lceil \log_2(|\Sigma| + k) \rceil$ битов.

LZW жил в GIF и Unix-утилите compress, пока LZ77+Хаффман не стал стандартом DEFLATE.

Шесть методов на одном слове

Сведём счёт на «ABRACADABRA», где исходно $11 \cdot 8 = 88$ битов.

Метод	Биты	Доля	Что использует
ASCII без сжатия	88	100%	ничего
Код Хаффмана	23	26%	неравные частоты
Арифметическое кодирование	23	26%	частоты без округления
LZ77	76	86%	ссылки в окно
LZ78	77	88%	словарь фраз
LZW	36	41%	только номера фраз

Тройки LZ77 стоят $3 + 3 + 8$ битов, пары LZ78 — $3 + 8$, номера LZW — 4.

Арифметическое сжатие

Хаффман округляет длину каждого символа до целого числа битов. Арифметическое кодирование округляет один раз на всё сообщение.

Определение 5 (Арифметическое кодирование)

При арифметическом кодировании всё сообщение кодируется одним числом — точкой отрезка $[0, 1)$.

Символ с вероятностью p_i сжимает текущий отрезок в p_i раз, и после n символов отрезок имеет длину $W = p_1 p_2 \cdots p_n$.

Достаточно любой двоичной дроби внутри отрезка — не более $\lceil -\log_2 W \rceil$ битов.

Число битов на символ сходится к энтропии, потому что округление до целого бита, за которое платит Хаффман, исчезает, а реальный кодер держит отрезок в 32- или 64-битных целых и отдаёт старшие биты по мере фиксации.

Арифметика на одном слове

Пример

Для «ABRACADABRA» частотами служат сами буквы, то есть $p_A = \frac{5}{11}$, $p_B = p_R = \frac{2}{11}$, $p_C = p_D = \frac{1}{11}$.

Длина финального отрезка $W = \left(\frac{5}{11}\right)^5 \left(\frac{2}{11}\right)^4 \left(\frac{1}{11}\right)^2$, и $-\log_2 W \approx 22.4$ бита — это nH.

Кодер выдаёт 23 бита, как и Хаффман, а вот на кривой монете $(0.9, 0.1)$ Хаффман тратит целый бит на символ, и только арифметика на длинном сообщении выходит на 0.47.

Арифметический кодер — блочный код Хаффмана, у которого блок вырос до всего сообщения, поэтому зазор $\frac{1}{n}$ из границы энтропии тает.

Бинарный вариант под именем CABAC работает в видеокодеках H.264 и H.265.

Отрезок на двух символах

Пример (Кодирование «AB»)

Возьмём те же частоты, $p_A = \frac{5}{11}$ и $p_B = \frac{2}{11}$, и сообщение «AB».

1. Первый символ A оставляет отрезок $\left[0, \frac{5}{11}\right)$.
2. Внутри него B оставляет $\left[0, \frac{10}{121}\right)$, и длина равна $W = \frac{10}{121} \approx 0.083$.
3. Дробь $\frac{1}{16} = 0.0001_2$ лежит внутри, поэтому четырёх битов хватает.

Декодер делит $\frac{1}{16}$ на $\frac{5}{11}$ и получает $\frac{11}{80} \approx 0.1375$, а это меньше $\frac{2}{11} \approx 0.182$, значит, второй символ — B.

На коротком сообщении арифметика проигрывает, ведь 4 бита больше двух, которые Хаффман тратит на те же два символа.

Выигрыш появляется на длине, где округление одного числа перестаёт быть заметным.

Кривая монета и блоки

Пример (Кривая монета)

$$H = -0.9 \log_2 0.9 - 0.1 \log_2 0.1 \approx 0.469,$$

то есть 0.47 бита на символ, и весь запас информации — в редком десятом.

Пример (Код на парах символов)

Пары источника имеют частоты $0.81, 0.09, 0.09, 0.01$.

Хаффман на них даёт $00 \to 1$, $10 \to 00$, $01 \to 011$, $11 \to 010$.

$$L = 0.81 \cdot 1 + 0.09 \cdot 2 + 0.09 \cdot 3 + 0.01 \cdot 3 = 1.29$$

бита на пару — 0.645 на символ.

Одиночный код тратит бит вместо 0.47, а блоки помогают, но таблица растёт как 2^n.

Пропускная способность канала

Определение 6 (Пропускная способность)

Канал переворачивает каждый бит с вероятностью p, и его пропускная способность равна

$$C = 1 - H(p) = 1 + p \log_2 p + (1-p) \log_2 (1-p).$$

p	C	Канал
0	1	без ошибок
0.1	0.53	умеренный шум
0.5	0	бесполезен
1	1	все биты перевёрнуты

При $p = 0.5$ принятый бит не зависит от переданного, и канал не несёт ничего.

Пример пропускной способности

Пример (Шум 10%)

$$H(0.1) = 0.1 \log_2 10 + 0.9 \log_2 (10/9) \approx 0.469,$$

поэтому $C = 1 - 0.469 = 0.531$.

Из тысячи переданных битов надёжно доходят примерно 531, а остальные 469 уходят на избыточность, которая оплачивает шум.

При $p = 1$ канал переворачивает все биты, и переворот известен заранее. Такой канал снова несёт бит на бит, поэтому $C = 1$.

Теорема Шеннона

Теорема 7 (Шеннон, 1948 — канал)

При любой скорости $R < C$ существуют коды с вероятностью ошибки, стремящейся к нулю.

При любой скорости $R > C$ надёжная передача невозможна.

Пример

При $p = 0.1$ и $C \approx 0.53$ скорость 0.4 достижима со сколь угодно малой ошибкой, а 0.6 — нет.

Доказательство показывает существование кодов, но не даёт явного построения, годного для практики.

Явные коды у границы строили семь десятилетий, вплоть до LDPC и полярных кодов.

Сжать, потом защитить

Защита и сжатие двигают одну величину — долю полезного в переданном.

Данные сжимают кодом Хаффмана, затем защищают кодом Хэмминга.

Сжатие убирает лишнюю избыточность, канальный код добавляет её назад — дозированно и там, где её ждёт декодер.

Порядок важен, и сначала идёт сжатие, а потом защита.

Если первым поставить код Хэмминга, он вставит проверочные биты в несжатый текст, и сжимать придётся уже перемешанные с избыточностью данные — выигрыша не будет.

Циклические коды

Определение 7 (Циклический код)

Линейный код называется **циклическим**, если вместе с каждым словом он содержит все его циклические сдвиги.

Сдвиг действует по правилу

$$(c_0, ..., c_{n-1}) \to (c_{n-1}, c_0, ..., c_{n-2}).$$

Слово — многочлен $c(x) = c_0 + c_1 x + ... + c_{n-1} x^{n-1}$, а сдвиг — умножение на x по модулю $x^n - 1$.

Кодовые слова — в точности кратные порождающему многочлену $g(x)$, который делит $x^n - 1$.

Пример циклического кода

Проверим сдвиги в коде длины 7 с порождающим многочленом $g(x) = 1 + x + x^3$.

Пример

Слово g записывается битами 1101000.

Сдвиг даёт 0110100, второй 0011010, третий 0001101.

Четвёртый сдвиг выносит член за край, ведь $x^4 g = x^4 + x^5 + x^7 \equiv 1 + x^4 + x^5$, то есть 1000110.

Слово осталось в коде, ведь $x^4 g = g(x^2 + x + 1)$ по модулю $x^7 - 1$.

Деление с остатком

При кодировании $c(x) = x^{n-k}m(x) - r(x)$, где $r(x)$ — остаток от деления на $g(x)$.

Пример (Код $(7, 4)$)

Порождающий многочлен $g(x) = 1 + x + x^3$ делит $x^7 - 1$, ведь

$$x^7 - 1 = (1 + x + x^3)(1 + x + x^2 + x^4).$$

Сообщение $m(x) = 1$ сдвигается в x^3, и деление даёт $x^3 = (1 + x + x^3) + (1 + x)$.

Частное равно 1, остаток $r = 1 + x$, а вычитание в поле из двух элементов совпадает со сложением, поэтому слово равно $x^3 + 1 + x = g$.

Декодирование повторяет то же деление, и ненулевой остаток служит сигналом об ошибке.

CRC в Ethernet

CRC — тот же остаток, применённый к пакету. Отправитель дописывает $r(x)$ в конец, а получатель делит принятое на $g(x)$.

Пример

CRC-32 в Ethernet ловит любую пакетную ошибку длиной до 32 битов — царапина короче четырёх байтов не пройдёт незамеченной.

Более длинные пачки ошибок тоже почти всегда видны, и мимо проходит лишь доля 2^{-32}.

CRC обнаруживает, но не исправляет, поэтому повреждённый пакет отбрасывается, а доставку берёт на себя транспортный уровень.

CRC-32 живёт в Ethernet, PNG и gzip.

Коды Рида — Соломона

Определение 8 (Код Рида–Соломона)

Код Рида — Соломона строят над полем $\mathrm{GF}(2^8)$. Сообщение $(m_0, \ldots, m_{k-1})$ задаёт многочлен степени меньше k.

Кодовое слово — значения этого многочлена в $n = 255$ степенях примитивного элемента поля.

Многочлен степени меньше k однозначно восстанавливается по любым k значениям — это интерполяция.

Порождающий многочлен $g(x) = (x - \alpha)(x - \alpha^2)\cdots(x - \alpha^{2s})$ делит $x^n - 1$, и RS-код — циклический с $2s = n - k$ проверочными символами.

Кодовое расстояние равно $d = n - k + 1$, ведь два разных многочлена совпадают не более чем в $k - 1$ точках.

Маленький код RS

Пример

$\mathrm{RS}(7, 3, 5)$ над $\mathrm{GF}(8)$, где $8 = 2^3$.

Три символа кодируются семью, расстояние $d = 7 - 3 + 1 = 5$.

Исправляет до $\frac{7-3}{2} = 2$ символьных ошибок, хотя каждый символ — 3 бита.

Две символьные ошибки — это до шести испорченных битов, а алгебра многочлена чинит их одной операцией над символами.

Пакетные ошибки

Царапина портит биты подряд.

Код Рида–Соломона считает ошибки символами, и испорченный байт чинится целиком, пока таких байтов не больше $\frac{n-k}{2}$.

Пример

У кода $\mathrm{RS}(255, 239)$ скорость $R = \frac{239}{255} \approx 0.94$.

Избыточность всего 6% — и до 8 байтовых ошибок исправляются.

RS-коды на практике

RS-коды выбирают там, где ошибки приходят пачками, а не поодиночке, потому что исправление идёт символами.

Царапина стирает целый участок, и побитовый код потратил бы исправления на один символ, а символьный чинит его целиком.

- QR-коды — RS над $\mathrm{GF}(256)$ с уровнями коррекции от 7% до 30% повреждённой площади.
- CD и DVD — каскад CIRC из двух слоёв RS с перемежением против царапин и пыли.
- Цифровое телевидение DVB — $\mathrm{RS}(204, 188)$ с исправлением до 8 байтовых ошибок на блок.
- RAID 6 — два проверочных диска, и данные выживают при отказе двух дисков разом.

Проверка сжатия

Вопросы охватывают построение кодов и словарные схемы.

Вопросы

1. Постройте код Хаффмана для $(0.4, 0.3, 0.2, 0.1)$ и посчитайте среднюю длину.
2. Почему код $\{0, 10, 110, 111\}$ префиксный, а $\{0, 01, 011\}$ — нет?
3. Проверьте неравенство Крафта для $(1, 2, 3, 3, 3)$ и постройте код для $(2, 2, 3, 3)$.
4. Для источника $(0.9, 0.1)$ код Хаффмана тратит бит при энтропии 0.47 — почему?
5. Сожмите «ABRACADABRA» тройками LZ77 и назовите тройку, которая кодирует финальное «ABRA».

Проверка границ и защиты

Вопросы о пределах сжатия и о кодах с исправлением ошибок.

Вопросы

1. Какую долю строк можно сжать на 10 битов, и почему gzip этому не противоречит?
2. Посчитайте энтропию распределения $\left(\frac{1}{2}, \frac{1}{4}, \frac{1}{4}\right)$.
3. Посчитайте пропускную способность канала с $p = 0.1$ и объясните, почему при $p = 1$ она снова равна единице.
4. Проверьте, что $g(x) = 1 + x + x^3$ делит $x^7 - 1$ над $\mathrm{GF}(2)$.
5. Сколько байтовых ошибок исправляет $\mathrm{RS}(255, 239)$ и почему он хорош против царапины на диске?

Что почитать?

Задача	Где читать
Строить префиксные коды	m13, раздел «Код Хаффмана (сжатие без потерь)»
Сжимать повторения	m13, раздел «Словарное сжатие: LZ77 и LZ78»
Понять предел передачи	m13, раздел «Граница Шеннона»
Проверить целостность	m13, раздел «Циклические коды»
Чинить пакетные ошибки	m13, раздел «Коды Рида–Соломона»